

Background

We currently have secrets stored in different places, mainly split across **Azure Key Vault** and **GitHub Secrets / Variables**. Now that HCP Vault is being introduced, we need to decide what should actually move to HCP Vault and what should continue to stay in Azure Key Vault.

The key focus: I don't think we should look at this as “*which vault is better?*”

The more useful question is: **What type of secret are we managing, and who actually needs to use it?**

What we already have in Key Vault

Looking at our current Key Vault, we have application and service credentials like:

```
postgresdb-password pecaMongoConnectionString snowflakePassword
snowflakePrivateKeyBase64 / snowflakePrivateKeyPassphrase ca-campaign-api-client-secret
storage-service-api-client-secret
```

These are mostly application/service credentials. My view is that there is no strong reason to move these just because HCP Vault is available. They can continue to live in Azure Key Vault.

Where HCP Vault comes into the picture

The main reason I see for bringing in HCP Vault is around **GitHub Actions secrets and variables**. Today, pipeline tools rely on direct repository secrets:

```
GitHub └─ Repository └─ Secrets ── SONAR_TOKEN ── NPM_TOKEN ──
ARTIFACTORY_TOKEN └─ DOCKER_TOKEN
```

These secrets are not really application secrets—they exist because the pipeline needs them. Instead of keeping these credentials directly in GitHub, GitHub Actions can authenticate to HCP Vault and pull required values dynamically during workflow execution.

Proposed Boundary Separation

Azure Key Vault

Use for: Secrets belonging to the application or Azure runtime environment.

- DB passwords & Mongo connection strings
- Snowflake credentials
- Application API credentials
- Certificates & Private keys

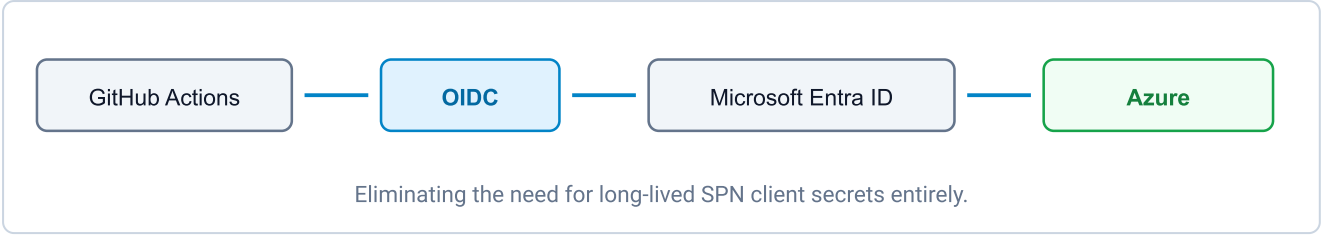
HCP Vault

Use for: Secrets belonging to CI/CD workflows and engineering platforms.

- Sonar & NPM tokens
- Artifact repository credentials
- Docker registry credentials
- Shared pipeline & CI/CD API tokens

What about SPN credentials?

This is something I think we should handle separately. If GitHub Actions currently relies on direct client secrets (`client_id`, `tenant_id`, `client_secret`), we can store the secret in Key Vault if necessary. However, for GitHub → Azure authentication, we should move toward **OIDC / federated identity**.



I wouldn't move an SPN secret from Key Vault to HCP Vault just because HCP Vault is being introduced. The first question should be whether we need the SPN secret at all.

What about Terraform-generated secrets?

Same logic applies here. Generating a password via Terraform and placing it straight into Key Vault for application use (Terraform → Azure Key Vault → Application) is completely fine. "Dynamically generated" shouldn't be the deciding factor—**where the secret is used** is what matters.

One Thing to Avoid: Duplicate Storage

Avoid having the exact same secret residing in both systems simultaneously. It introduces friction:

- Which one is the true source of truth?
- Who is responsible for rotation, and what happens when values diverge?
- Which team owns it, and where do we audit access/changes?

Proposed Rule & Decision Matrix

If the secret is mainly used by...	Keep it in...	Notes
Application / Azure Workload	Azure Key Vault	DBs, certificates, app secrets
Database / App Integration	Azure Key Vault	Mongo, Snowflake, internal APIs
GitHub Actions / CI/CD Tooling	HCP Vault	Sonar, NPM, Docker, shared pipelines
Non-sensitive Configuration	GitHub Variables / Config	Standard environment settings
GitHub → Azure Authentication	Prefer OIDC	Avoid static credentials where possible

The Main Point

